E-COMMERCE

Customer Cohort & Retention Analysis

A Production-Ready Analytics Platform — Architecture, Tech Stack & Build Plan

Prepared as a client-ready technical project document

Version 1.0 · June 2026

Table of Contents

1. Executive Summary

This document specifies the end-to-end build of a Customer Cohort & Retention Analysis platform for an e-commerce business. The platform ingests order and customer-event data, computes cohort retention, churn, repeat-purchase rate, and customer lifetime value (LTV), and presents the results through an interactive dashboard that a non-technical stakeholder (founder, marketing lead, or ops manager) can use without touching SQL.

The goal is a deliverable that looks and feels like a real commercial analytics product — fast, visually consistent, and trustworthy — not a one-off Jupyter notebook. Every layer below (data, backend, frontend, design system) is chosen for production use, not just prototyping.

> **What "done" looks like**
>
> A stakeholder logs into a web dashboard, picks a date range and cohort granularity (weekly/monthly), and sees retention curves, a cohort heatmap, churn rate, repeat purchase rate, and LTV by acquisition channel — all loading in under 2 seconds, on a consistent design system, with exportable reports.

Who this is for

Freelance/contract delivery to a real e-commerce client (Shopify, WooCommerce, or custom-platform merchant) who wants recurring visibility into customer retention rather than a static one-time report.

Core deliverables

- A deployed, authenticated web dashboard (not just local scripts)
- A documented, reproducible data pipeline (raw orders → cohort tables)
- A defined set of retention/cohort metrics with explicit formulas
- A consistent UI design system (not ad-hoc styling per page)
- A handoff package: README, ERD, API docs, deployment guide

2. Problem Definition & Scope

2.1 Business Problem

E-commerce businesses generally know their revenue but not their retention. Acquisition cost is easy to track; what's hard is answering: "Of the customers we acquired in March, how many are still buying in June?" and "Which acquisition channel produces customers who actually stick around?" Without cohort-level retention visibility, marketing spend optimizes for first purchase, not long-term value — which is frequently the wrong objective.

2.2 In Scope

- Cohort construction by first-purchase month/week
- Retention curves (% of cohort purchasing again in period N)

— Churn rate and repeat purchase rate by cohort and by channel
— Customer Lifetime Value (historical and predicted, simple model)
— RFM segmentation (Recency, Frequency, Monetary) as a complementary view
— Filterable dashboard: date range, channel, product category, geography
— Scheduled data refresh (daily batch) and manual refresh trigger
— Role-based access (admin vs. viewer) and exportable PDF/CSV reports

2.3 Out of Scope (v1)

— Real-time/streaming event tracking (clickstream) — batch is sufficient for retention analysis
— Marketing automation or triggered campaigns based on churn prediction
— Multi-tenant SaaS support — this is a single-client deployment for v1
— Advanced ML churn prediction beyond a baseline logistic/gradient-boosted model

3. Technology Stack

The stack below favors current, well-supported, production-grade tools that are also fast to build and cheap to host for a single-client engagement. Each choice includes the reasoning so you can swap components if the client's environment dictates otherwise (e.g., they're already on AWS, or already pay for Snowflake).

3.1 Stack at a Glance

Layer	Technology	Why
Frontend	Next.js 15 (React 19, App Router) + TypeScript	SSR for fast first load, file-based routing, best-in-class DX in 2026
UI System	Tailwind CSS v4 + shadcn/ui + Radix primitives	Consistent, accessible components; design tokens enforce visual consistency
Charts	Recharts (primary) + visx (custom cohort heatmap)	Recharts for standard charts; visx for the bespoke retention-grid visual
State / Data fetching	TanStack Query (React Query) v5	Caching, background refetch, loading/error states out of the box
Backend API	FastAPI (Python 3.12)	Async, auto-generated OpenAPI docs, natural fit with the pandas/SQL analytics layer
Data transformation	dbt (data build tool) + SQL	Version-controlled, testable, documented SQL transformations — industry standard
Database	PostgreSQL 16	Reliable, window-function support (critical for cohort SQL), cheap to host

Layer	Technology	Why
Orchestration	Dagster (or cron + Python for v1 simplicity)	Schedules and monitors the daily ETL; Dagster scales better than cron later
Auth	Auth.js (NextAuth) or Clerk	Drop-in auth with role-based access, avoids hand-rolled session security
Hosting — Frontend	Vercel	Zero-config Next.js deploys, previews per branch, fast CDN
Hosting — Backend/DB	Railway or Render + managed Postgres (Supabase/Neon)	Low-ops managed hosting appropriate for a single-client project
Containerization	Docker + docker-compose (local dev)	Reproducible local environment matching production
CI/CD	GitHub Actions	Lint, test, type-check, and deploy on push to main
Monitoring	Sentry (errors) + Better Stack/Logtail (logs)	Catch pipeline failures and frontend errors before the client does

3.2 Why Not a BI Tool Instead?

A fair question is: why not just use Metabase, Looker, or Power BI? For a freelance engagement, a custom dashboard is justified when the client wants something white-labeled, embeddable in their own portal, or when the metrics (cohort retention curves, RFM, LTV) need custom logic that off-the-shelf BI tools handle clumsily. If the client is fine with a generic BI tool's look and you want to ship faster, swap Section 3's frontend/backend stack for a dbt + Metabase combo and skip straight to Section 5's data model — the analytics logic is identical either way.

4. System Architecture

4.1 High-Level Flow

```
STEP 4   dbt Models
         (staging --> marts: cohort, rfm, ltv)
              |
              v
STEP 5   Analytics Marts
         (Postgres: queryable cohort/rfm/ltv tables)
              |
              v
STEP 6   FastAPI
         (REST/JSON endpoints, no business logic)
              |
              v
STEP 7   Dashboard
         (Next.js, charts + filters + tables)
```

4.2 Component Responsibilities

Component	Responsibility
Ingestion	Pulls orders, customers, and line items from the source platform (Shopify Admin API, WooCommerce REST API, or scheduled CSV drop) into a raw Postgres schema, unmodified.
dbt staging models	Clean column types, standardize timestamps to UTC, deduplicate, handle refunds/cancellations consistently.
dbt mart models	Build cohort tables, RFM scores, and LTV aggregates as queryable tables/views the API reads directly — no logic lives in the API layer.
FastAPI service	Thin layer: validates filters (date range, channel, segment), queries marts, returns typed JSON. No business logic — that lives in dbt SQL, where it's tested and version-controlled.
Next.js dashboard	Renders filters, charts, and tables; calls the API via TanStack Query; handles auth via Auth.js middleware.
Orchestrator	Triggers ingestion → dbt run daily at a fixed time; alerts on failure via Sentry/Slack webhook.

5. Data Model

5.1 Core Entities (Raw Layer)

Table	Key Columns
customers	customer_id (PK), email_hash, first_seen_at, acquisition_channel, country, signup_source

Table	Key Columns
orders	order_id (PK), customer_id (FK), order_date, status, total_amount, currency, discount_amount
order_items	order_item_id (PK), order_id (FK), product_id, category, quantity, unit_price
refunds	refund_id (PK), order_id (FK), refund_date, refund_amount
marketing_spend (optional)	date, channel, spend_amount — enables blended CAC alongside retention

5.2 Analytics Marts (dbt Output)

- mart_cohort_retention — one row per (cohort_month, period_number): cohort_size, active_customers, retention_rate
- mart_customer_rfm — one row per customer: recency_days, frequency_count, monetary_total, rfm_segment
- mart_customer_ltv — one row per customer: historical_ltv, predicted_ltv_12mo, acquisition_channel
- mart_channel_performance — one row per (channel, cohort_month): cohort_size, blended_cac, ltv_to_cac_ratio

5.3 Cohort Retention — the Core SQL Logic

This is the single most important query in the project. It assigns each customer to a cohort by the month of their first order, then for every subsequent calendar month measures what fraction of that cohort placed at least one order.

```
with first_orders as (
  select customer_id, date_trunc('month', min(order_date)) as cohort_month
  from {{ ref('stg_orders') }}
  group by 1
),
activity as (
  select customer_id, date_trunc('month', order_date) as activity_month
  from {{ ref('stg_orders') }}
  group by 1, 2
),
joined as (
  select
    f.cohort_month,
    a.activity_month,
    datediff('month', f.cohort_month, a.activity_month) as period_number,
    a.customer_id
  from activity a
  join first_orders f using (customer_id)
)
select
```

```
  cohort_month,
  period_number,
  count(distinct customer_id) as active_customers
from joined
group by 1, 2
order by 1, 2;
```

This output is then divided by cohort_size (the count of customers in period_number = 0) to produce the retention_rate used in the heatmap and curve charts — that division happens in a downstream dbt model, not in the API, so it's testable in isolation.

6. Metrics Definitions

Precise definitions matter — "retention" and "churn" are calculated differently by different teams. These are the definitions this project commits to; document them visibly in the dashboard (e.g., a tooltip or methodology page) so the client never has to guess what a number means.

Metric	Definition
Cohort	All customers whose first purchase occurred in the same calendar month (or week, if weekly granularity is selected).
Retention Rate (period N)	% of a cohort's original customers who placed at least one order in calendar period N after their first purchase.
Churn Rate	1 − Retention Rate for a given period; alternatively, % of customers with no order in the trailing 90 days (rolling churn).
Repeat Purchase Rate	% of all customers (any cohort) who have placed 2 or more orders, over a selected date range.
Customer LTV (historical)	Sum of net revenue (after refunds) per customer to date.
Customer LTV (predicted)	Historical LTV extrapolated using a simple BG/NBD or Gamma-Gamma model (lifetimes library), or, for v1, average order value × predicted order count from a basic regression.
RFM Segment	Customer scored 1–5 on Recency, Frequency, Monetary, then bucketed into named segments (e.g., "Champions", "At Risk", "Lost").
LTV:CAC Ratio	Customer LTV divided by blended customer acquisition cost for their channel/cohort — needs marketing_spend data.

7. Dataset Strategy

Since you don't have client data yet, build and demo the platform against a realistic public dataset first, then swap in the client's real export once contracted. This de-risks the build — UI, dbt models, and API are all validated before the client's data ever touches the system.

7.1 Recommended Public Datasets

Dataset	Source	Why it fits
Olist Brazilian E-Commerce	Kaggle — olistbr/brazilian-ecommerce	Real orders, customers, products, reviews, geolocation; ~100k orders; ideal scale for cohort analysis
Online Retail II (UCI)	UCI Machine Learning Repository	Classic transactional dataset (UK retailer, 2009–2011); good for RFM/cohort, widely used so easy to validate logic against known benchmarks
Instacart Market Basket	Kaggle — instacart-market-basket-analysis	Strong for repeat-purchase and basket-level behavior, less ideal for monetary LTV (no real prices)
Synthetic generator (Faker + custom logic)	Self-generated	Use if you need to control cohort shapes precisely for demoing specific retention patterns to the client

7.2 Recommendation

> **Use Olist for the build-out demo**
>
> It has the closest shape to a real e-commerce client's order export (customers, orders, payments, items, geography, delivery), is large enough to show meaningful cohort curves, and is well-known enough that a client or interviewer can sanity-check your numbers against public Olist analyses.

7.3 Getting Real Client Data Later

- Shopify: Admin API (orders.json, customers.json) or a scheduled CSV export from Shopify Reports
- WooCommerce: WooCommerce REST API (wc/v3/orders, wc/v3/customers)
- Generic platforms: request a raw orders + customers CSV export — map columns to the raw schema in Section 5.1

Because dbt staging models isolate source-specific quirks from the marts, switching data sources later only requires rewriting the staging layer, not the cohort/RFM/LTV logic itself.

8. UI/UX Design System

"Smooth and consistent" comes from a design system, not from styling each page individually. Define tokens once, build components against them, and every screen inherits consistency automatically.

8.1 Design Tokens

Token	Value / Rule
Primary color	Indigo/blue (#2E5BFF) — used for primary actions, active filters, key chart series
Neutral palette	Slate scale (Tailwind 'slate') for text, borders, backgrounds — never pure black/white
Semantic colors	Green = positive/retained, Amber = at-risk, Red = churned — used consistently across every chart
Typography	Inter or Geist for UI text; tabular figures (font-variant-numeric: tabular-nums) for all metric tables so numbers align
Spacing scale	Tailwind's default 4px base scale — no arbitrary pixel values in components
Corner radius	Consistent rounded-xl (12px) on cards, rounded-md (6px) on inputs/buttons — one scale, applied everywhere
Elevation	Two shadow levels only (resting card, hovered/active) — avoid ad-hoc box-shadows
Motion	150–200ms ease-out transitions on hover/filter changes; skeleton loaders (not spinners) while data fetches

8.2 Core Screens

1. Overview Dashboard — top-line KPIs (active customers, repeat purchase rate, blended churn, average LTV) as cards, plus the cohort retention heatmap front and center.
2. Cohort Explorer — full retention curve chart (one line per cohort) with toggles for weekly/monthly granularity and channel filter.
3. RFM Segments — scatter or treemap of customers by segment, with drill-down to a customer list table.
4. Channel Performance — LTV:CAC comparison table/bar chart by acquisition channel.
5. Methodology — static page documenting every metric's exact formula (ties back to Section 6), so the client always trusts the numbers.

8.3 Consistency Rules

- Build a shared <MetricCard>, <FilterBar>, <ChartContainer>, and <DataTable> component used on every page — no page invents its own card or table styling
- One global filter bar (date range, channel, segment) persists across all screens via URL query params, so filters don't reset when navigating
- Every chart shares one color mapping for cohorts/segments — a cohort that's blue on the heatmap is the same blue on the retention curve
- Loading, empty, and error states are designed once per component type and reused everywhere — never a one-off blank screen

— Use shadcn/ui as the base so accessibility (focus states, keyboard nav, ARIA) comes for free instead of being retrofitted

9. Build Plan & Milestones

Phase	Duration	Deliverable
1. Setup	2–3 days	Repo scaffolding (Next.js + FastAPI + dbt + Docker), CI pipeline, Postgres schema, Olist data loaded into raw tables
2. Data layer	4–5 days	dbt staging + mart models (cohort, RFM, LTV) with dbt tests (uniqueness, not-null, referential integrity) passing
3. API	3–4 days	FastAPI endpoints for cohort retention, RFM, LTV, channel performance; OpenAPI docs; basic auth
4. UI foundation	3 days	Design tokens in Tailwind config, shared components (MetricCard, FilterBar, ChartContainer, DataTable)
5. Dashboard screens	6–8 days	All five core screens (Section 8.2) wired to live API data via TanStack Query
6. Polish & auth	3 days	Role-based access, PDF/CSV export, loading/error/empty states, responsive layout pass
7. Client data swap	2–4 days	Rewrite staging models for the client's real export; validate numbers against their existing reports
8. Deploy & handoff	2 days	Production deploy (Vercel + Railway/Render), monitoring wired up, README + ERD + API docs delivered

Total estimate: roughly 4–5 weeks part-time for a single freelancer, assuming the Olist build-out happens first and the client data swap (Phase 7) is scoped separately once their actual data export is in hand.

10. Handoff Package

— GitHub repository with clear folder structure: /frontend, /backend, /dbt, /infra

— README with local setup instructions (docker-compose up, seed data, run dbt)

— Entity-relationship diagram (ERD) of raw and mart tables

— API documentation (auto-generated via FastAPI's OpenAPI/Swagger UI)

— Methodology page (in-app) documenting every metric formula

— Deployment runbook: environment variables, hosting accounts, how to trigger a manual data refresh

— A short Loom/video walkthrough of the dashboard for non-technical stakeholders

11. Reference Links

Next.js Documentation

shadcn/ui Components

FastAPI Documentation

dbt Documentation

Olist Brazilian E-Commerce Dataset (Kaggle)

Online Retail II Dataset (UCI)

TanStack Query Documentation